SECURE HOSPITAL-TO-PHARMACY SYNCHRONIZATION SYSTEM

Submitted by

TAMILARASAN M

1P24MC054

In partial fulfillment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

From Bharathiar University, Coimbatore.

Under the internal supervision of

Dr.J.GNANA MANO SHEEBHA, M.C.A., M. Phil., NET.,

Assistant Professor

SCHOOL OF COMPUTER STUDIES

MASTER OF COMPUTER APPLICATIONS

RVS COLLEGE OF ARTS AND SCIENCE (AUTONOMOUS)

Sulur, Coimbatore – 641402.

March - 2026

RVS COLLEGE OF ARTS AND SCIENCE (AUTONOMOUS)

SCHOOL OF COMPUTER STUDIES -MCA

Master Of Computer Applications





March- 2026

Register Number: 1P24MC054

Certified bona fide Project Work done by TAMILARASAN M

Guide **Director**

Submitted for the Project Evaluation and *Viva voce* held on _____

Internal Examiner External Examiner

Certificate

CERTIFICATE

This is to certify that the project work titled **SECURE HOSPITAL-TO-PHARMACY SYNCHRONIZATION SYSTEM**, submitted to the School of Computer Studies-MCA, RVS College of Arts and Science, in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications is a record of original project work done by **TAMILARASAN M** during the period Nov 2025 to Mar 2026 towards the partial fulfillment of the requirement for the award of MCA prescribed by RVS College of Arts and Science, Sulur.

Internal Supervisor

Declaration

DECLARATION

I am **TAMILARASAN M**, hereby declare that the project entitled **SECURE HOSPITAL-TO-PHARMACY SYNCHRONIZATION SYSTEM** submitted to the School of Computer Studies, RVS College of Arts and Science, in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications is a record of original project work done by me during the period November 2025 to March 2026 under the internal supervision of **Dr.J.GNANA MANO SHEEBHA, M.C.A., M. Phil., NET., Assistant Professor, RVS College of Arts and Science (Autonomous)** From Bharathiar University, Coimbatore.

Signature of the Candidate

Acknowlegement

ACKNOWLEDGEMENTS

I express my sincere thanks to our Managing Trustee **Dr. K. Senthil Ganesh MBA (USA)., MS (UK)., Ph.D.,** for providing us with the adequate faculty and laboratory resources for completing my project successfully.

I express my sincere thanks to our Secretary Madam **Prof. Saramma Samuel.,** for providing us with the guidance for completing my project successfully.

I take this as a fine opportunity to express my sincere thanks to **Dr. T. Sivakumar M.Sc., M. Phil., Ph.D., Principal,** RVS College of Arts and Science (Autonomous) for giving me the opportunity to undertake this project.

I express my sincere thanks to **Dr. P. Navaneetham M.Sc., M.Phil., Ph.D., Director (Administration), Department of Computer Science** for the help and advice throughout the project.

I express my sincere thanks to **Dr. S. Yamini M.Sc., (CC)., M. Phil., Ph.D., Director (Academic), Department of Computer Science** for her valuable guidance and prompt correspondence throughout the curriculum to complete the project.

I am grateful to **Dr. G. Purusothaman, M.Sc. (IT), M.Phil. (CS), Ph.D. (CS), Associate Professor &Head, School of Computer Studies (PG)**, for providing this opportunity and for his constant encouragement

I express my gratitude to **Dr.J.GNANA MANO SHEEBHA, M.C.A., M. Phil., NET., Assistant Professor** for her valuable guidance, support, encouragement and motivation rendered by her throughout this project.

Finally, I express my sincere thanks to all other staff members and my dear friends, and all dear and near for helping me to complete this project.

TAMILARASAN M

Secure Hospital-to-Pharmacy Synchronization System

[MEDISYNC]

`

Abstract

ABSTRACT

MediSync is a secure healthcare management platform designed to improve communication and data synchronization between hospitals, doctors, pharmacies, and patients. Developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), the system provides a modern and efficient solution for managing digital prescriptions and healthcare workflows. The platform enables hospitals to manage doctors and patients, doctors to generate secure electronic prescriptions, pharmacies to verify prescription authenticity before dispensing medicines, and patients to access their prescription records and healthcare notifications.

The application allows hospitals to register doctors, allocate departments and room numbers, and link patients to specific doctors for consultation. Doctors can view assigned patients and create prescriptions that are digitally stored in the system. Pharmacies can retrieve prescriptions and verify them before dispensing medicines, ensuring authenticity and preventing prescription fraud. Patients can log in to their dashboard to view assigned doctors, receive notifications, and track their active prescriptions.

On the backend, MediSync uses Node.js and Express.js to provide a secure RESTful API that interacts with MongoDB for flexible and scalable data management. The system implements secure authentication using JWT (JSON Web Tokens) and password encryption to protect user accounts. Additionally, the platform includes role-based access control, audit logging for tracking sensitive actions, and notification systems for improved communication between healthcare stakeholders.

With a responsive user interface developed using React and modern UI frameworks, MediSync ensures a seamless experience across multiple devices. The platform aims to digitize the prescription workflow, reduce medical errors, prevent prescription fraud, and improve overall healthcare coordination between hospitals and pharmacies.

Contents

CONTENTS

Certificate	I
Declaration	II
Acknowledgements	III
Abstract	IV

CHAPTER 1

1. Introduction	1
1.1 An overview of the project	1
1.2 Mission of the project	3
1.3 Background study	3
1.3.1 A study of the existing system	4

CHAPTER 2

2. System Analysis	5
2.1 A study of the proposed system	5
2.2 User requirement specification	6
2.2.1 Major modules	7
2.2.2 Sub modules	8
2.3 Software requirement specification	9
2.4 System specification	10
2.4.1 React.js	10
2.4.2 Node.js	11
2.4.3 Hardware configuration	11
2.4.4 Software configuration	11

CHAPTER 3

3. System design and development	12
3.1 Fundamentals of design concepts	12
3.1.1 Abstraction	13
3.1.2 Refinement	13
3.1.3 Modularity	13

3.2 Design Notations 14
3.2.1 System structure chart 14
3.2.2 System flow diagram 15
3.2.3 Data flow diagram / UML 16
3.2.4 Software Engineering Model 18
3.3 Design process 19
3.3.1 Software Architecture 19
3.3.2 Control Hierarchy 20
3.3.3 Structural Partitioning 20
3.3.4 Data structure 20
3.4 Table design 20
3.5 Input design 21
3.6 Output design 21
3.7 Development approach 21

CHAPTER 4

4. Testing and implementation 22
4.1 Testing 22
4.1.1 System testing 22
4.1.2 White box testing 22
4.1.3 Unit testing 23
4.2 System implementation 25
4.2.1 System maintenance 26

CHAPTER 5

5. Conclusion 27
5.1 Directions for future enhancements references 27

BIBLIOGRAPHY 29

ANNEXURE

ANNEXURE - A - Output design 33
ANNEXURE - B - Source code 38

Introduction

1. INTRODUCTION

In modern healthcare systems, effective coordination between hospitals, doctors, pharmacies, and patients is essential for delivering safe and reliable medical services. Traditional prescription management methods rely heavily on handwritten prescriptions and manual verification processes, which are often prone to errors, misinterpretation, and fraudulent activities. These challenges can lead to incorrect medication dispensing, lack of transparency in prescription verification, and difficulty in maintaining accurate medical records. Furthermore, pharmacies frequently face challenges in verifying whether a prescription has genuinely been issued by an authorized medical professional. The absence of a secure and centralized digital system creates communication gaps between hospitals and pharmacies, increases administrative workload, and compromises patient safety. To address these issues, the healthcare sector requires a secure digital platform that can streamline prescription management, ensure authenticity, and provide a reliable communication channel between healthcare stakeholders.

1.1 An overview of the project

The **MediSync – Secure Hospital-to-Pharmacy Synchronization System** is a modern web-based healthcare platform developed using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**. The system is designed to digitize the prescription workflow and establish a secure communication network between hospitals, doctors, pharmacies, and patients.

The platform enables hospitals to manage doctors, assign patients to specific medical professionals, and monitor prescription activities within their organization. Doctors can consult patients and generate secure digital prescriptions that are stored within the system. Pharmacies can retrieve prescriptions and verify their authenticity before dispensing medication, ensuring that the prescriptions are issued by authorized doctors.

The system allows patients to:

- View their assigned doctor information including department, room number, floor details.
- Access their active and previous prescriptions securely.
- Receive notifications when prescriptions are created or updated.
- Track prescription status and medication details.

Doctors can:

- View patients assigned to them by the hospital administration.
- Create and manage secure digital prescriptions.
- Maintain accurate digital records of patient consultations.

Hospitals can:

- Register and manage doctors within the system.
- Allocate departments, rooms, and floors for doctors.
- Register patients and link them with specific doctors.
- Monitor prescription activities and hospital operations.

Pharmacies can:

- Retrieve prescriptions from the system.
- Verify prescription authenticity before dispensing medicines.
- Update prescription status after medication is dispensed.

By providing a centralized and automated healthcare management platform, MediSync enhances coordination between healthcare providers, improves prescription security, and reduces the risks associated with manual prescription handling. The system promotes transparency, improves patient safety, and ensures efficient healthcare service delivery.

1.2 Mission of the Project

The mission of the MediSync – Secure Hospital-to-Pharmacy Synchronization System is to provide a secure, efficient, and user-friendly digital platform that improves coordination between hospitals, doctors, pharmacies, and patients. The system aims to replace traditional paper-based prescription management with a modern automated solution that ensures authenticity, improves communication, and enhances patient safety. By digitizing the prescription workflow, the platform helps healthcare providers maintain accurate medical records, verify prescriptions securely, and streamline the medication dispensing process.

The key objectives of this project are to:

- Reduce manual effort in prescription management and healthcare record maintenance.
- Improve communication and coordination between hospitals, doctors, pharmacies, patients.
- Prevent prescription fraud through secure digital prescription verification.
- Enable hospitals to manage doctors, patients, and pharmacy relationships efficiently.
- Provide patients with easy access to their prescriptions and healthcare notifications.
- Ensure secure authentication and role-based access control for all users.
- Maintain a centralized database for healthcare records and system activities.

Through these objectives, the project aims to build a reliable and modern healthcare management platform that enhances efficiency, transparency, and security in medical prescription handling.

1.3 Background Study

In many healthcare institutions, prescription management and communication between hospitals, doctors, and pharmacies are still handled through traditional manual systems. Doctors often write prescriptions on paper, which are then handed over to patients and presented at pharmacies for medication dispensing. These manual processes are prone to several issues such as prescription forgery, misinterpretation of handwritten instructions, loss of prescription documents, and lack of verification mechanisms.

These approaches often result in:

- Prescription forgery and misuse of handwritten prescriptions
- Misinterpretation of medication instructions due to unclear handwriting
- Delays in verifying prescriptions at pharmacies
- Inefficient management of patient records and prescription history
- Lack of a centralized system to track prescriptions issued by doctors

Many healthcare institutions still do not have a unified digital system that allows hospitals, doctors, pharmacies, and patients to securely share prescription information and verify medication authenticity in real time. A well-designed platform like MediSync can effectively address these issues by providing a centralized and secure digital environment that improves communication, ensures prescription verification, and enhances coordination among healthcare stakeholders.

1.3.1 A Study of the Existing System

The existing prescription management system in many healthcare institutions is largely manual and presents several limitations:

- Prescriptions are handwritten, which can lead to misinterpretation of medication instructions.
- Pharmacies often lack a reliable mechanism to verify the authenticity of prescriptions.
- Hospitals maintain patient and prescription records manually, leading to data redundancy and inefficiency.
- Patients must physically carry prescriptions, which can be lost or damaged.
- Communication between hospitals and pharmacies is limited and not digitally integrated.
- There is no centralized system to monitor prescription activities and maintain secure records.
- Manual processes increase administrative workload and reduce operational efficiency.

These limitations highlight the need for a secure and automated healthcare management system that can improve prescription verification, enhance communication between healthcare stakeholders, and maintain reliable digital medical records.

\

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

System analysis is the process of examining the existing healthcare workflow and prescription management methods to identify their limitations and determine the requirements for a more efficient system. In many healthcare environments, prescriptions are still handled through manual processes such as handwritten prescriptions and physical record maintenance. These traditional methods often lead to issues such as prescription forgery, misinterpretation of medical instructions, lack of proper verification mechanisms, and inefficient communication between hospitals, doctors, pharmacies, and patients..

The goal of system analysis in this project is to study the current manual prescription handling process and identify its weaknesses, including delays in verification, difficulty in maintaining centralized records, and lack of transparency in the prescription lifecycle. These issues highlight the need for a secure and automated healthcare management system that can streamline communication and ensure the authenticity of prescriptions.

The proposed **MediSync – Secure Hospital-to-Pharmacy Synchronization System** is designed to address these challenges by introducing a centralized digital platform. The system enables hospitals to manage doctors and patients, doctors to generate secure electronic prescriptions, pharmacies to verify prescription authenticity before dispensing medicines, and patients to access their prescription details and healthcare notifications.

By integrating all healthcare stakeholders into a single platform, the system improves communication, enhances prescription security, reduces administrative workload, and ensures efficient healthcare coordination.

2.1 A Study on the Proposed System

The proposed system is a modern web-based healthcare management platform that automates the prescription management process and improves communication between hospitals, doctors, pharmacies, and patients. The system is designed to replace manual prescription handling with a secure digital workflow that ensures authenticity, transparency, and efficient management of medical records.

The MediSync platform allows hospitals to manage their medical staff and patients through a centralized interface. Hospitals can register doctors, assign departments and room allocations, and link patients with specific doctors for consultations. Doctors can access the system to view assigned patients and generate digital prescriptions that are securely stored in the database.

The system allows:

- Hospitals to manage doctors, patients, and pharmacy relationships within the healthcare network.
- Doctors to create and manage secure digital prescriptions for their assigned patients.
- Pharmacies to retrieve prescriptions and verify their authenticity before dispensing medication.
- Patients to access their prescriptions, view assigned doctor details, and receive healthcare notifications.

By implementing this system, MediSync enhances prescription verification, improves communication between healthcare providers, and provides a secure and reliable digital platform for managing healthcare operations using the MERN stack architecture.

2.2 User Requirement Specification

The User Requirement Specification (URS) defines the functional and non-functional requirements of the MediSync platform from the perspective of its users. It ensures that the system satisfies the operational needs of hospitals, doctors, pharmacies, patients, and administrators while maintaining a secure and reliable healthcare management environment. The specification outlines the essential system functionalities and performance expectations required to support secure prescription management and healthcare coordination.

Functional Requirements:

- Hospitals should be able to register doctors, assign departments, room numbers, and manage patient records.
- Doctors should be able to view assigned patients and generate secure digital prescriptions for medical treatment.
- Pharmacies should be able to retrieve prescriptions from the system and verify their authenticity before dispensing medication.

- Patients should be able to access their prescription history, view assigned doctor details, and receive healthcare notifications.

- Administrators should be able to manage system users, approve hospital and pharmacy registrations, and monitor system activities through audit logs.

Non-Functional Requirements:

- The system should provide a user-friendly and responsive interface that is accessible through modern web browser.

- The system must ensure high data security through JWT-based authentication and encrypted password storage.
- The platform should maintain reliable communication between hospitals, doctors, pharmacies, and patients through a centralized database.

- The system should support efficient performance and handle multiple user requests simultaneously without affecting system stability.

2.2.1 Major Modules

The **MediSync – Secure Hospital-to-Pharmacy Synchronization System** is divided into five major modules to efficiently manage the roles and responsibilities of different users within the healthcare platform.

1. Admin Module

Allows administrators to manage the overall system operations. The admin can approve hospital and pharmacy registrations, monitor system activities, manage user accounts, and review audit logs to ensure system security and transparency.

2. Hospital Module

Enables hospitals to manage doctors and patients within their organization. Hospitals can register doctors, assign departments and room numbers, register patients, link patients to specific doctors, and monitor healthcare activities.

3. Doctor Module

Allows doctors to view patients assigned to them, review patient information, and generate secure digital prescriptions. Doctors can manage prescription records and provide treatment instructions through the platform.

4. Pharmacy Module

Provides pharmacies with the ability to retrieve prescriptions issued by doctors and verify their authenticity before dispensing medicines. Pharmacies can also update prescription status once medication has been dispensed.

5. Patient Module

Allows patients to access their prescriptions, view assigned doctor details including department and room information, and receive notifications related to prescriptions and healthcare updates.

2.2.2 Sub Modules

Each major module of the system is further divided into sub-modules to perform specific tasks efficiently and ensure smooth healthcare coordination among users.

1. Admin Sub-Modules:

- **User Approval Management:** Administrators can approve or reject hospital and pharmacy registrations before allowing them access to the platform.
- **System Monitoring:** Enables administrators to monitor system activity, review logs, and track important operations performed by users.
- **User Account Management:** Allows administrators to manage user accounts and maintain overall system integrity.

2. Hospital Sub-Modules:

- **Doctor Management:** Hospitals can register doctors, assign departments, room numbers, and manage doctor information within the hospital network.
- **Patient Registration:** Hospitals can register new patients and maintain their medical details in the system.

- **Patient–Doctor Linking:** Hospitals can assign patients to specific doctors for consultation and treatment.

3. Doctor Sub-Modules:

- **Patient Management:** Doctors can view patients assigned to them and review patient details before consultation.
- **Prescription Creation:** Doctors can generate secure digital prescriptions for patients including medication details and dosage instructions.
- **Prescription History:** Doctors can view previously issued prescriptions and maintain digital medical records.

4. Pharmacy Sub-Modules:

- **Prescription Verification:** Pharmacies can verify prescriptions before dispensing medication to ensure authenticity.
- **Medicine Dispensing:** Pharmacies can update prescription status once medicines have been dispensed to patients.

5. Patient Sub-Modules:

- **Prescription Viewer:** Patients can view their current and previous prescriptions through the system dashboard.
- **Doctor Information:** Patients can check assigned doctor details including department, room number, and floor location.
- **Notification System:** Patients receive notifications when prescriptions are created or when healthcare updates are available.

2.3 Software requirement specification

System Overview:

The **MediSync – Secure Hospital-to-Pharmacy Synchronization System** is a web-based healthcare platform developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to automate prescription management and improve coordination between hospitals, doctors, pharmacies, and patients.

The system provides a centralized digital environment where hospitals can manage doctors and patients, doctors can generate secure electronic prescriptions, pharmacies can verify prescription authenticity before dispensing medicines, and patients can access their prescription records and healthcare notifications.By replacing traditional paper-based prescriptions with a secure digital system, MediSync improves communication, enhances transparency, and ensures reliable record management within the healthcare ecosystem.

Problem Statement:

In many healthcare institutions, prescriptions are still managed through handwritten documents and manual verification processes. These traditional methods often lead to issues such as prescription forgery, misinterpretation of medical instructions, loss of prescription records, and lack of proper communication between hospitals and pharmacies. Pharmacies frequently struggle to verify whether a prescription has genuinely been issued by an authorized doctor, which can result in incorrect medication dispensing and compromise patient safety. Additionally, hospitals face challenges in managing doctor assignments, patient records, and prescription tracking efficiently. To address these challenges, a centralized digital platform like MediSync is required to automate prescription workflows, ensure secure verification, maintain reliable medical records, and improve communication among healthcare stakeholders.

2.4 System Specification

The system specification describes the technical components required for the development and deployment of the **MediSync – Secure Hospital-to-Pharmacy Synchronization System**. The platform is built using the MERN stack architecture to ensure efficient handling of healthcare data, secure communication between users, and reliable management of digital prescriptions. The system integrates modern web technologies to provide a scalable, responsive, and secure healthcare management environment.

2.4.1 React.js

React.js is used as the frontend framework for developing the user interface of the MediSync platform. It enables the creation of dynamic, responsive, and interactive web pages for different users such as administrators, hospitals, doctors, pharmacies, and patients.

Its component-based architecture allows the development of reusable UI components such as dashboards, prescription forms, and notification panels. React ensures efficient state management and smooth user interaction, providing a modern and user-friendly experience across all modules of the system.

2.4.2 Node.js & Express.js

Node.js functions as the server-side runtime environment responsible for handling backend logic and processing user requests. Together with Express.js, it provides a robust framework for building RESTful APIs that manage authentication, prescription processing, user management, and database communication. The backend system also ensures secure authentication using JWT and handles communication between hospitals, doctors, pharmacies, and patients through well-defined API endpoints.

2.4.3 Hardware configuration

- Processor: Intel i3 (12th Gen) or higher
- RAM: Minimum 8 GB
- Storage: 250 GB HDD/SSD or above
- Network: Stable high-speed internet connection

2.4.4 Software configuration

- Operating System: Windows, Linux, or macOS
- Frontend: Frontend: React.js with modern UI libraries (e.g., Tailwind CSS)
- Backend: Node.js with Express.js
- Database: MongoDB (Document-oriented NoSQL)
- Authentication: JWT (JSON Web Token) and password encryption
- Development Tools: Visual Studio Code and Node Package Manager (npm)

Design Modules

3. SYSTEM DESIGN AND IMPLEMENTATION

System design is the technical solution for the development of the **MediSync – Secure Hospital-to-Pharmacy Synchronization System**. This phase focuses on transforming the requirements identified during the system analysis stage into a structured and functional system architecture. The design process ensures that all components of the system work together efficiently to provide secure prescription management and reliable communication between hospitals, doctors, pharmacies, and patients.

The system design for MediSync is divided into two primary phases:

Logical Design:

Defines the healthcare data flow and system processes, such as patient-doctor assignment, prescription creation, verification by pharmacies, and notification handling, without specifying the physical implementation.

Physical Design:

Transforms the logical design into technical specifications including MongoDB database schemas, RESTful API structures, authentication mechanisms using JWT, and secure data communication between the React frontend and Node.js backend.

3.1 Fundamentals of Design Concepts

The design of the MediSync web application focuses on creating a secure and efficient healthcare management platform. The system architecture supports seamless communication between hospitals, doctors, pharmacies, and patients while ensuring data security and integrity. Key objectives include:

- Creating an intuitive dashboard interface for hospitals to manage doctors and patients.
- Implementing a secure prescription management system where doctors can generate digital prescriptions.
- Designing a verification mechanism that allows pharmacies to validate prescriptions before dispensing medication.
- Ensuring secure communication between the React frontend and the Node.js backend using JWT-based authentication.
- Maintaining a centralized MongoDB database to store healthcare records, prescriptions, and system logs.

3.1.1 Abstraction

In MediSync, abstraction is applied by modeling complex healthcare processes into simplified software components. Entities such as **Doctors, Patients, Prescriptions, and Pharmacies** are represented as database objects with clearly defined attributes and relationships. For example, a **Prescription** is treated as an abstract entity containing information such as patient details, doctor identity, medication instructions, and verification status. The internal complexity of data storage and authentication is hidden from users, allowing the system to focus on delivering efficient healthcare services.

3.1.2 Refinement

Refinement in MediSync follows a **top-down design approach**, where high-level system functions are progressively divided into smaller and more manageable processes. For example, the main function **Prescription Management** is refined into several steps such as **Doctor Authentication, Patient Selection, Prescription Creation, Pharmacy Verification, and Prescription Status Update**. This step-by-step breakdown helps ensure that every process is clearly defined and implemented efficiently within the system.

3.1.3 Modularity

The MediSync system is designed using a modular architecture to improve maintainability and scalability. The application is divided into independent modules such as **Authentication Module, Hospital Management Module, Doctor Module, Pharmacy Module, and Patient Module**. Each module performs specific tasks while interacting with other modules through well-defined APIs. This modular structure ensures high cohesion within each module and reduces dependency between system components, making the platform easier to maintain and extend in the future.

3.2 Design Notations

3.2.1 System Structure Chart

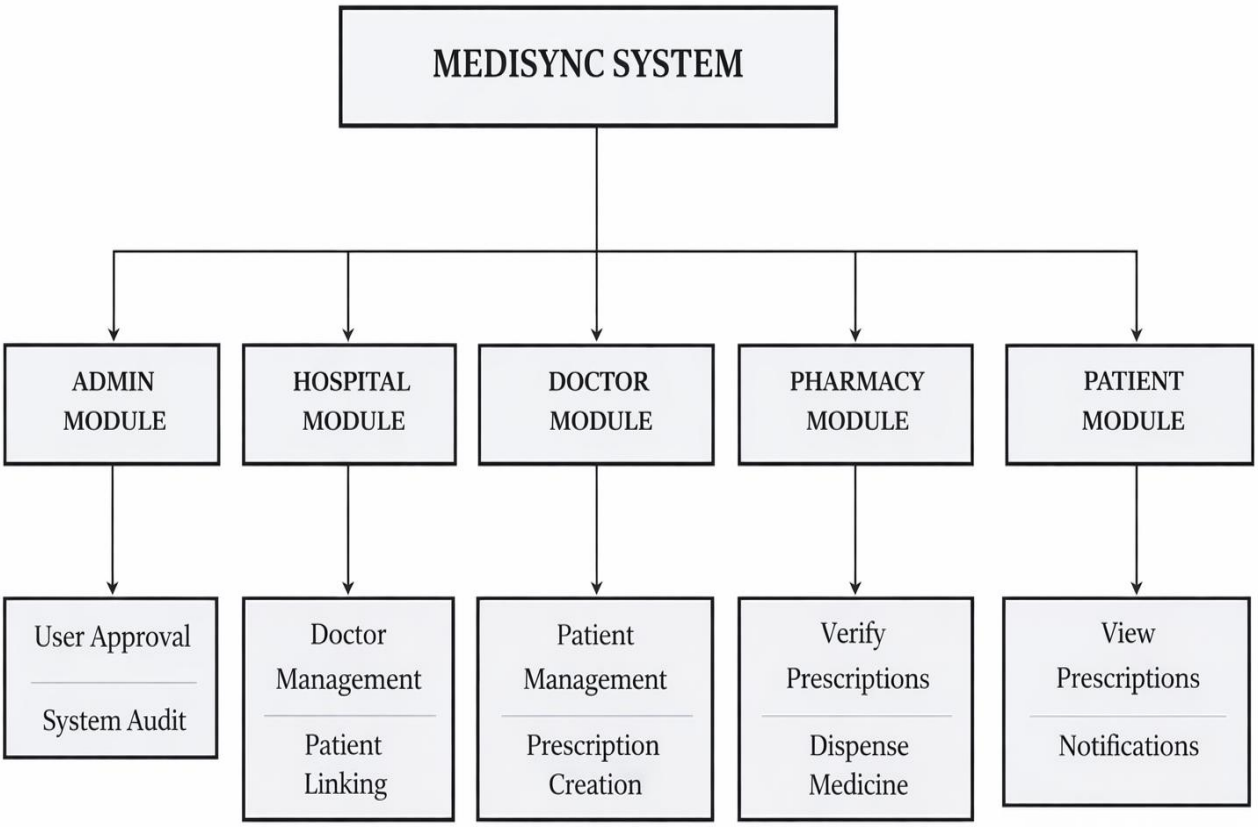


Figure 3.1 - System structure chart

3.2.2 System Flow Diagram

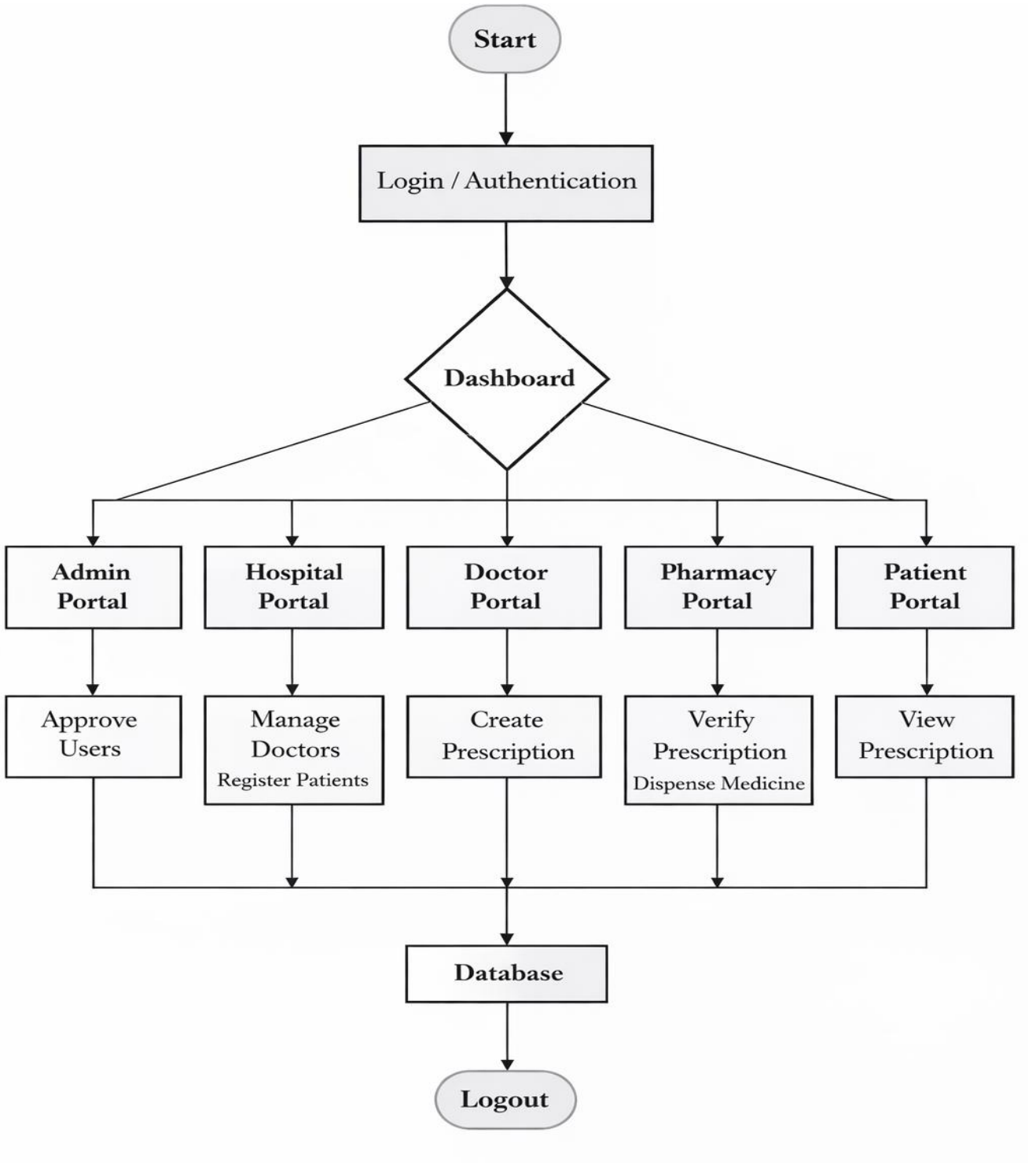


Figure 3.2 - System flow diagram

Illustrates the end-to-end operational workflow of the MediSync healthcare system, starting from login authentication to role-based healthcare operations and system logout.

3.2.3 Data Flow Diagram / UML

A Data Flow Diagram (DFD) for the MediSync system illustrates the movement of healthcare data between different users and the centralized database. It represents how information such as prescriptions, patient records, and user authentication details move through the system. The diagram helps in understanding how data is processed, stored, and transmitted between hospitals, doctors, pharmacies, patients, and the system administrator.

The DFD simplifies complex system interactions by showing how healthcare information flows through different processes such as patient registration, doctor assignment, prescription creation, and prescription verification.

EXTERNAL ENTITY

Key entities in the MediSync system include the **Admin, Hospital, Doctor, Pharmacy, and Patient**. These entities interact with the system to perform different healthcare operations such as managing users, creating prescriptions, verifying medicines, and accessing medical records.

PROCESS

Important processes in the MediSync system include **User Authentication, Doctor Management, Patient Registration, Prescription Creation, Prescription Verification, and Notification Management**. These processes ensure that healthcare data is properly processed and stored within the system.

DATA FLOW

Data flows such as **User Credentials, Patient Records, Prescription Data, Verification Requests, and Notification Messages** represent the movement of information across the system. These data flows connect different modules and ensure that all healthcare stakeholders receive accurate and up-to-date information within the MediSync platform.

LOGIN DFD:

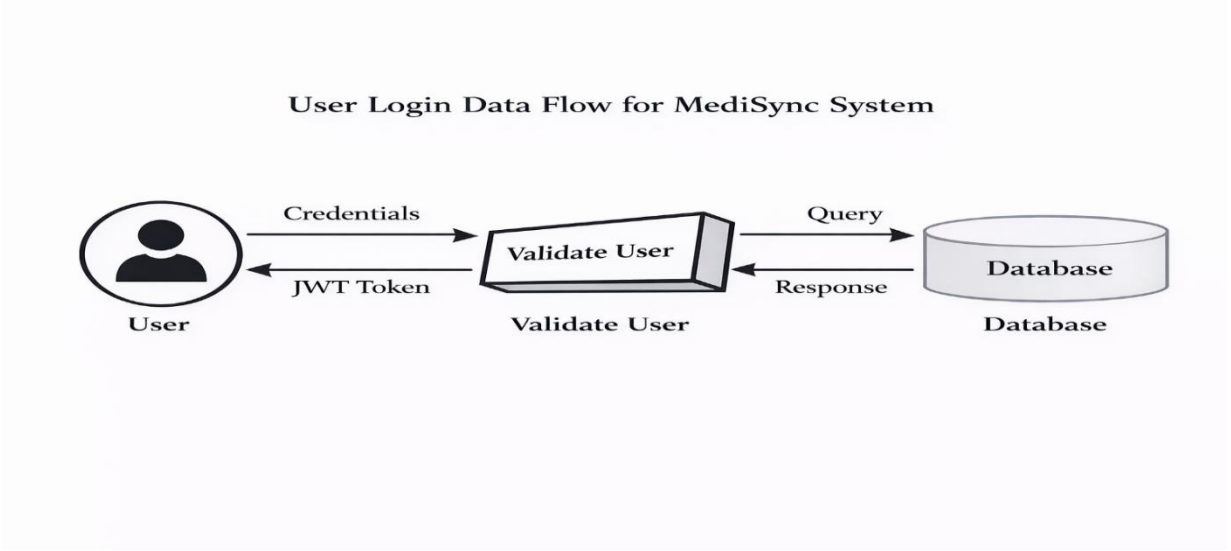


Figure 3.3

Shows the user authentication process where system users (Admin, Hospital, Doctor, Pharmacy, and Patient) submit their credentials to the Validate User process, which verifies the information against the MongoDB User collection and returns a JWT token for secure access to the MediSync system.

REGISTRATION DFD:

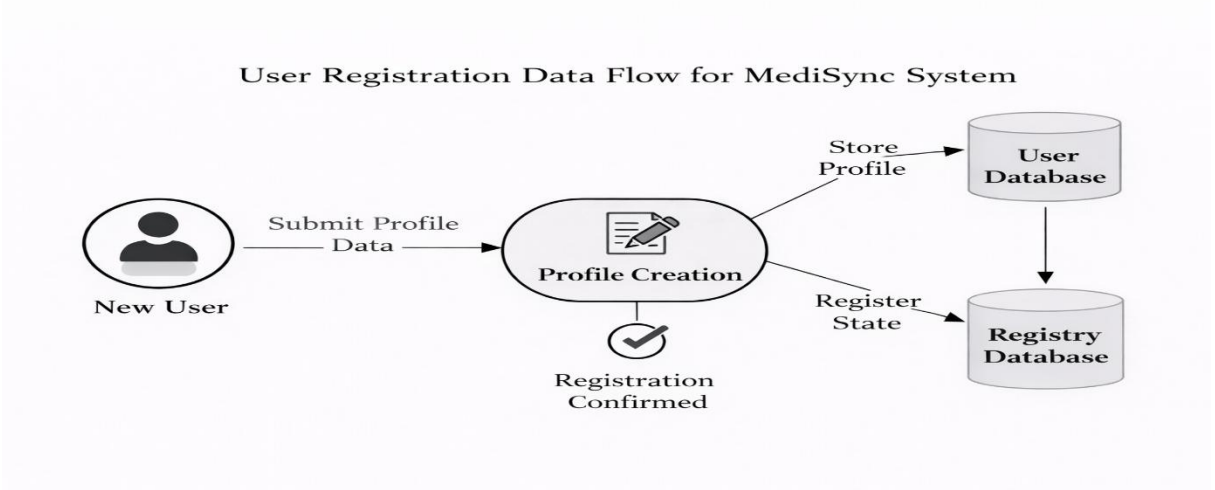


Figure 3.4

Shows the process of creating a new user profile in the MediSync system where a new user (Hospital, Pharmacy, or Patient) submits registration details to the Create Profile process, which stores the information in the User Database and maps the user role and status in the Registry Database.

ADMIN DFD:

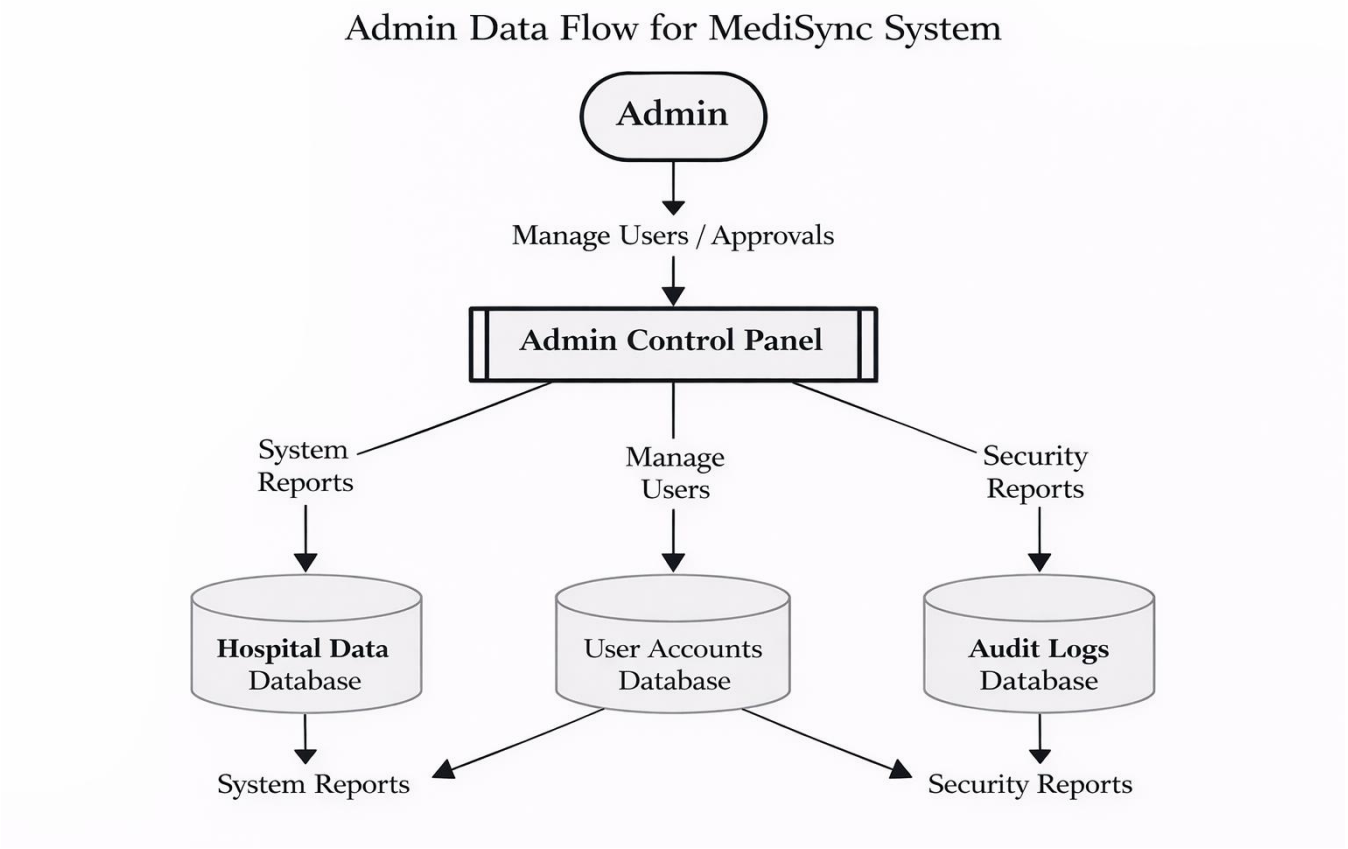


Figure 3.5

Shows administrative control over system users, hospital and pharmacy approvals, and monitoring of system-wide activities through audit logs and reports.

3.2.4 Software Engineering Model

The MediSync system follows an **Iterative and Incremental Software Development Model**, where the system is developed in multiple stages. Initially, core features such as user authentication, hospital registration, doctor management, and patient records are implemented. In later iterations, additional features like digital prescription creation, pharmacy verification, notifications, and system monitoring are added. Each cycle includes planning, analysis, design, implementation, and testing, allowing continuous improvement and refinement of the MediSync healthcare platform.

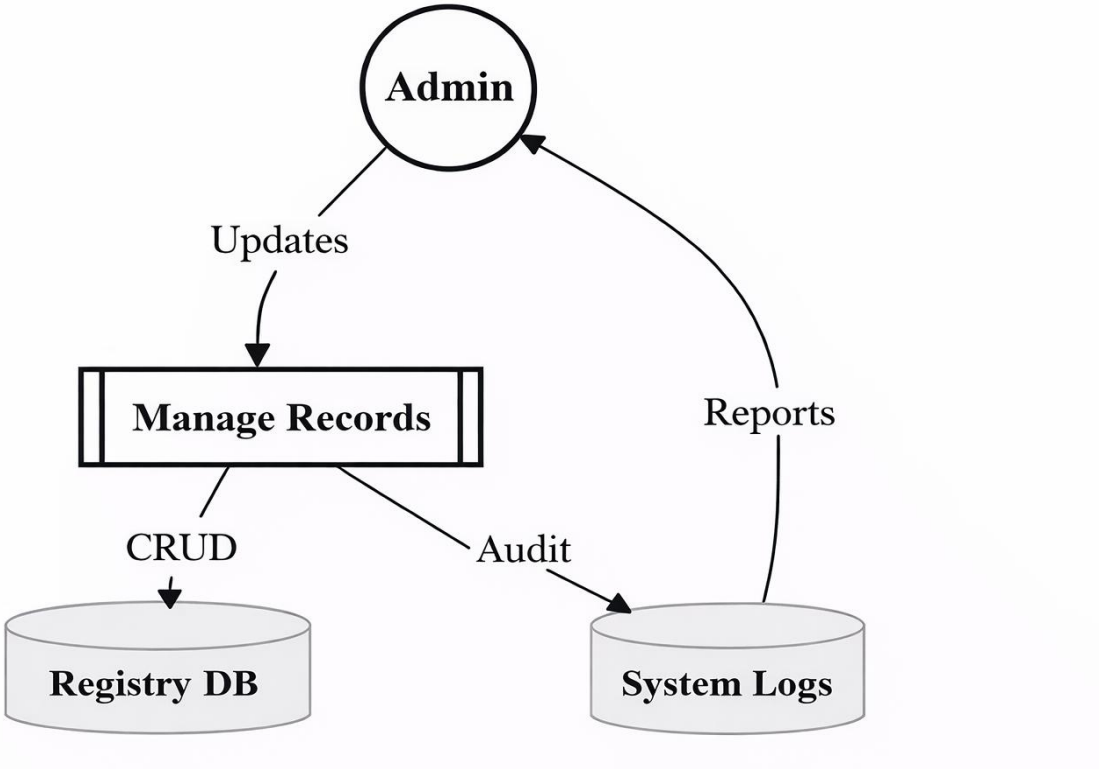


Figure 3.6 – Iterative Software Development Model

Illustrates the cyclic development process consisting of planning, analysis, design, implementation, and testing phases used to develop and refine the MediSync system.

3.3 Design Process

3.3.1 Software Architecture

The software architecture of MediSync is based on the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**. This architecture follows a Model-View-Controller (MVC) structure where React manages the user interface, Node.js and Express handle the server logic and API requests, and MongoDB stores healthcare data such as users, patients, prescriptions, and system logs.

3.3.2 Control Hierarchy

The control hierarchy of MediSync is organized into three layers. The **Top Layer (User Interface)** provides dashboards for Admin, Hospital, Doctor, Pharmacy, and Patient. The **Middle Layer (REST APIs)** processes system logic such as authentication, prescription creation, and verification. The **Bottom Layer (Database Layer)** manages the storage and retrieval of healthcare records in MongoDB.

3.3.3 Structural Partitioning

MediSync uses structural partitioning to separate different system functionalities. The system is divided into modules such as **Authentication, Hospital Management, Doctor Module, Pharmacy Module, and Patient Module**. This modular design improves maintainability, scalability, and system performance.

3.3.4 Data Structure

Efficient data structures are used to store healthcare information. User accounts, hospital details, patient records, and prescription data are stored in structured MongoDB collections. **JWT tokens** are used for secure session management, ensuring safe communication between the frontend and backend.

3.4 Table Design

The MediSync database contains several important collections including **Users, Hospitals, Patients, Prescriptions, and Audit Logs**. Each collection stores specific system data while maintaining relationships between doctors, patients, hospitals, and pharmacies to ensure efficient data management.

3.5 Input Design

Input design focuses on reducing user errors while entering healthcare information. Forms are designed for **user registration, patient registration, doctor assignment, and prescription creation**. Validation mechanisms ensure that all required information is entered correctly before data is stored in the database.

3.6 Output Design

The system provides outputs such as **patient prescription details, doctor assignments, pharmacy verification status, and administrative reports**. Information is displayed through user dashboards and structured tables, allowing users to easily view healthcare data and system updates.

3.7 Development Approach

The MediSync system follows an **Agile development approach** where the project is developed in iterative stages. Version control tools such as **Git** are used to manage source code changes. Each module undergoes testing before integration to ensure system stability, security, and reliable healthcare service management.

Testing and Implementation

4. TESTING AND IMPLEMENTATION

4.1 Testing

Software testing is an essential process used to identify errors and ensure that the MediSync system operates correctly according to its requirements. It serves as a final evaluation phase of the software development lifecycle, verifying the functionality of the system after design and implementation. For the MediSync healthcare platform, testing ensures that critical features such as user authentication, hospital management, prescription creation, and pharmacy verification operate reliably under different conditions.

4.1.1 System Testing

System testing of MediSync involves testing the complete integrated system to ensure that all modules work together properly. This includes testing the full workflow from user registration and login to prescription creation, verification by pharmacies, and patient access to prescription records. System testing ensures that the React frontend, Node.js backend, and MongoDB database interact correctly and maintain data consistency across the platform.

4.1.2 White Box Testing

White box testing focuses on verifying the internal logic and structure of the application. In MediSync, this testing approach is used to check the internal operations of modules such as authentication logic, prescription processing, and database communication. By examining the source code, developers ensure that all internal functions perform according to the specified system requirements.

4.1.3 Unit Testing

Unit testing focuses on testing individual components or modules of the system independently. In MediSync, modules such as the User Authentication Module, Doctor Management Module, Prescription Creation Module, and Pharmacy Verification Module are tested separately to ensure they function correctly. Unit testing helps identify bugs early in the development process and improves the reliability and stability of the system.

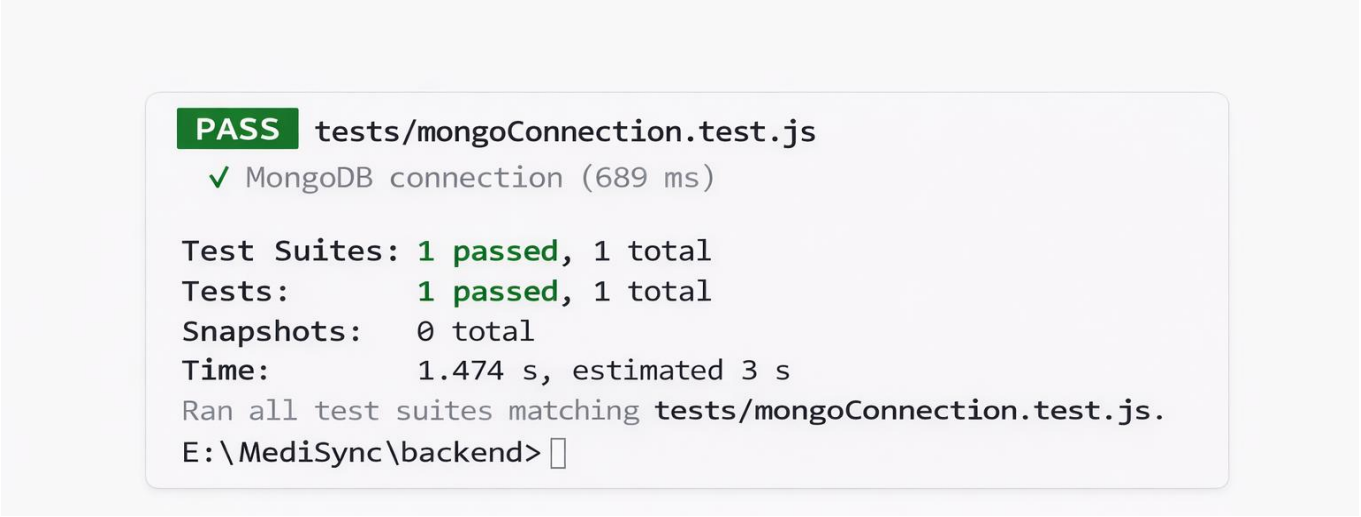


Figure 4.1

Unit testing result showing a successful MongoDB database connection in the MediSync backend using the Jest testing framework.

4.1.4 API Testing Results (Postman)

API testing is performed to verify that the backend endpoints of the MediSync system function correctly and that data is properly stored, retrieved, and updated in the MongoDB Atlas cloud database. Postman is used to send HTTP requests to the REST API endpoints and validate responses such as status codes, JSON outputs, authentication tokens, and database updates. These tests ensure that the communication between the frontend React application and the Node.js backend operates reliably.

POST : http://localhost:5000/api/auth/register

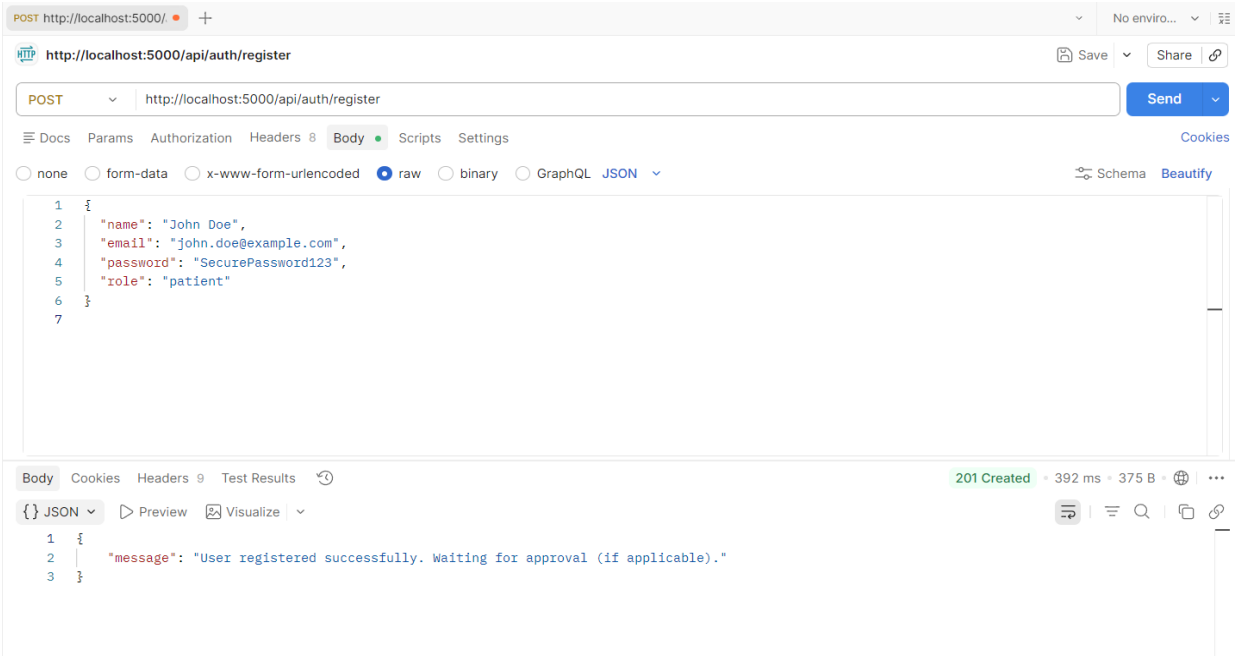


Figure 4.2 – API Testing: Successful User Registration (201 Created)

Shows the successful execution of the user registration API where a new user account is created in the MediSync system and the backend returns a **201 Created** status after storing the user details in the MongoDB database.

POST : http://localhost:5000/api/auth/login

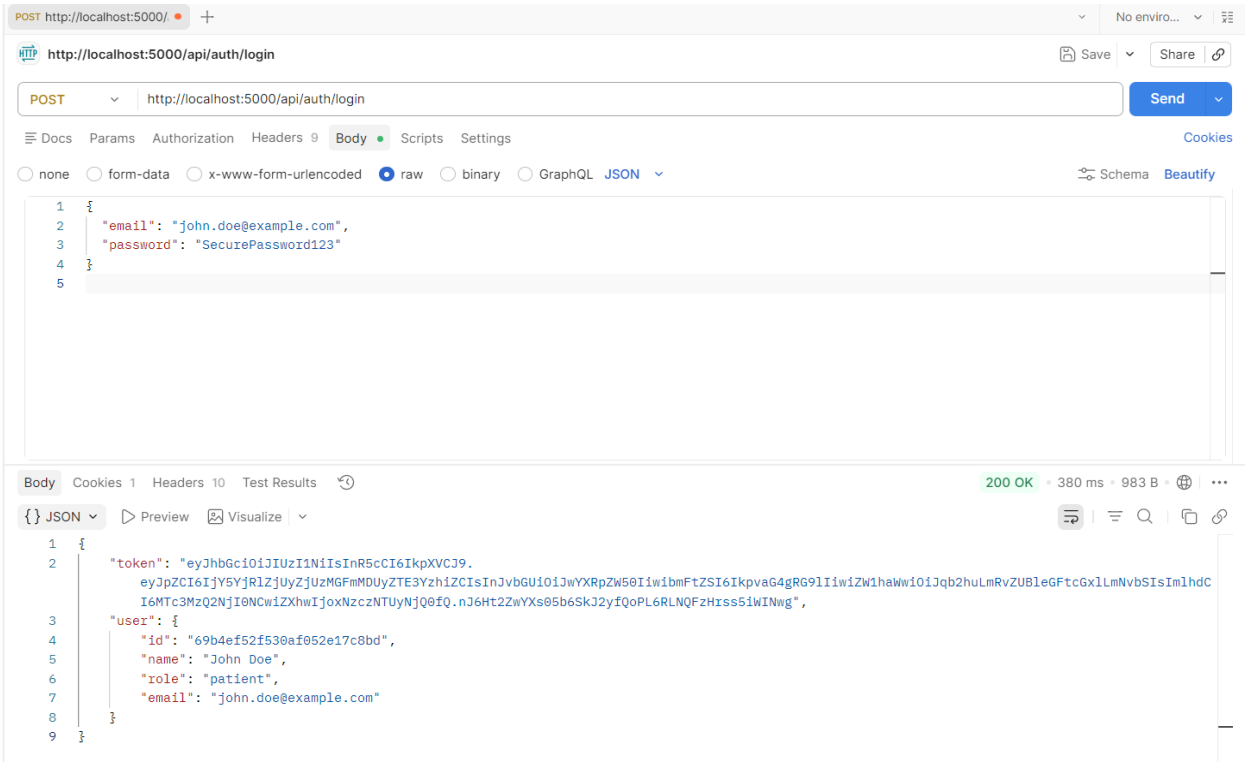


Figure 4.3 – API Testing: User Authentication and JWT Token Generation Shows the successful login API response where the system validates user credentials and generates a **JWT token** used for secure authentication and access to protected routes in the MediSync system.

GET : http://localhost:5000/api/prescriptions

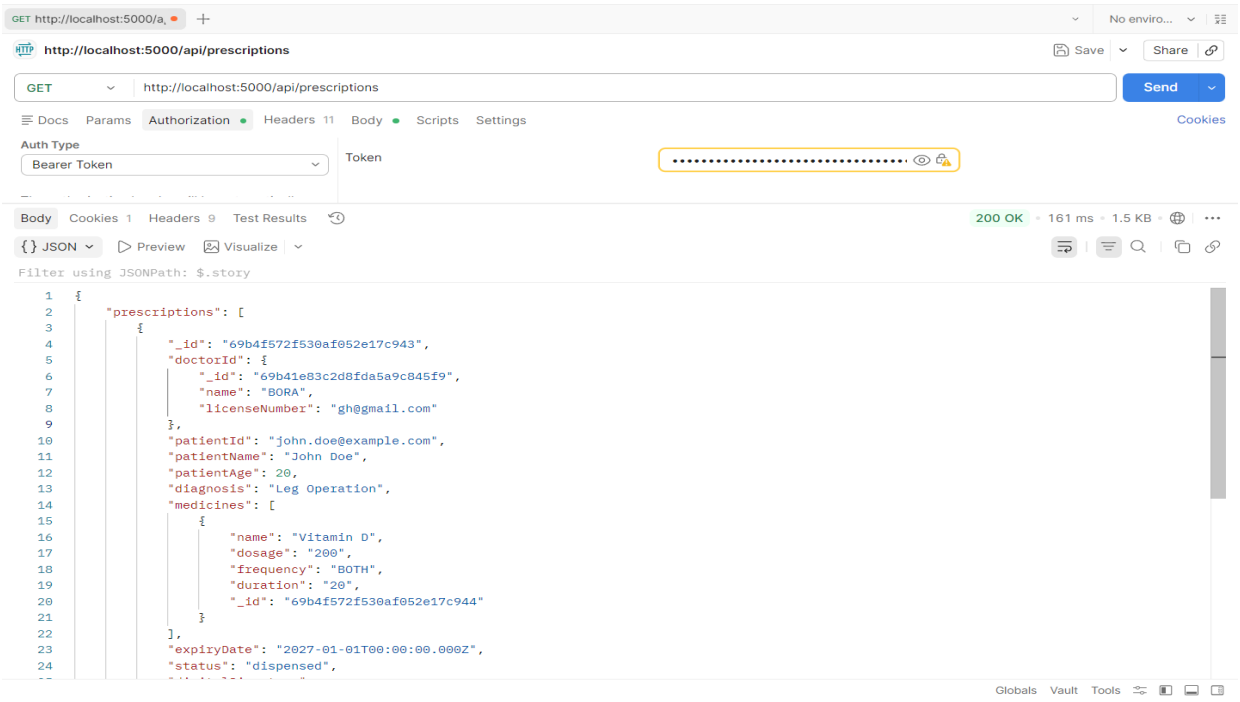


Figure 4.4 – API Testing: Retrieval of User Profile Data from Backend

This figure shows the successful retrieval of user profile information from the MediSync backend. The request includes a valid JWT token in the authorization header, allowing the system to verify the user's identity and return profile details stored in the MongoDB database.

4.2 System Implementation

System implementation is the phase where the theoretical design of the MediSync system is converted into a fully functional application. This stage involves deploying the backend services, configuring the database, and integrating the frontend interface with the backend APIs. The MediSync platform is developed using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**.

The backend server is configured to handle authentication, prescription management, and user role operations, while **MongoDB Atlas** is used as the cloud database to store user records, patient data, prescriptions, and system logs. The implementation ensures secure communication between modules and provides reliable access for Admin, Hospital, Doctor, Pharmacy, and Patient users.

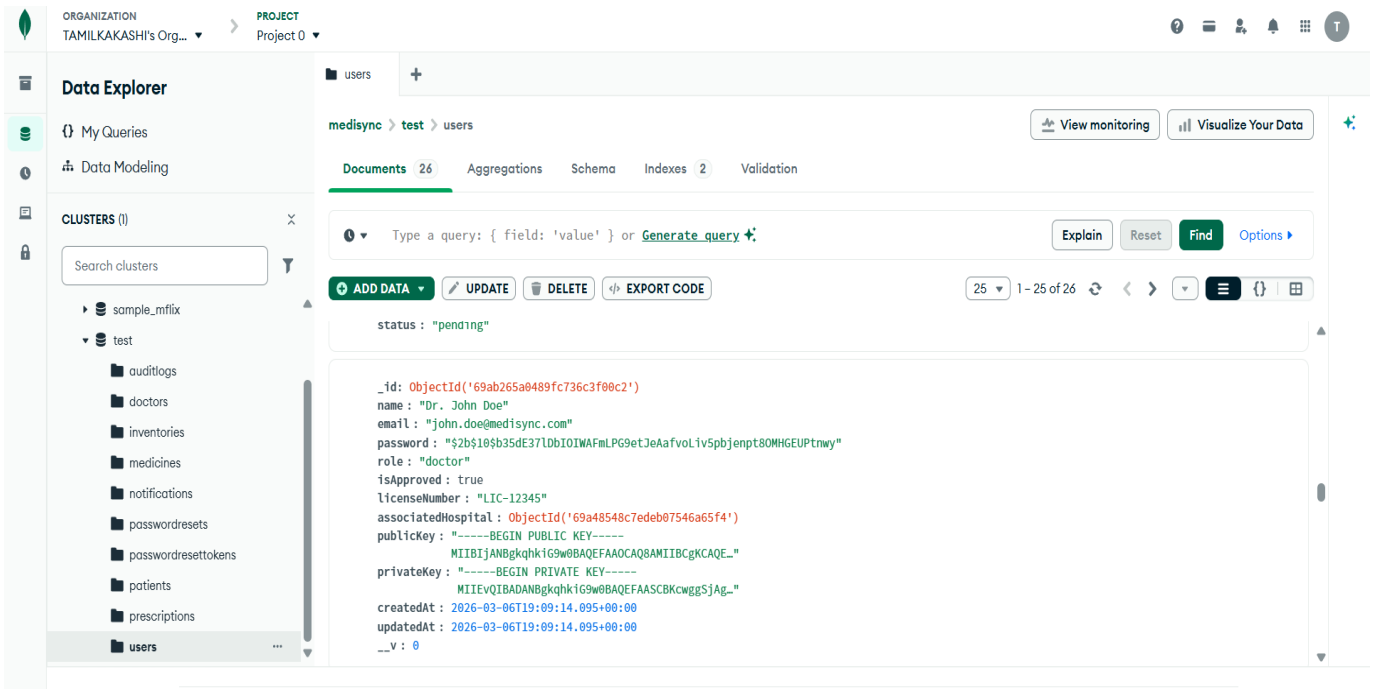


Figure 4.5 – Cloud Database Implementation (MongoDB Atlas Collections View)

4.2.1 System Maintenance

System maintenance is an important process that ensures the MediSync platform continues to operate efficiently and securely after deployment. Maintenance activities include **updating system dependencies**, monitoring backend server performance, maintaining database integrity, and resolving any technical issues that arise during system usage. Regular updates are applied to Node.js packages to fix security vulnerabilities and improve performance. Additionally, MongoDB Atlas databases are monitored and backed up periodically to prevent data loss. System logs are also reviewed to detect failed login attempts, API errors, and unusual system activities, ensuring the reliability and security of the MediSync healthcare platform

Conclusion

5. CONCLUSION

In conclusion, the development of the **MediSync – Secure Hospital-to-Pharmacy Synchronization System** using the MERN stack (MongoDB, Express.js, React.js, and Node.js) provides a modern and efficient healthcare management platform. The system establishes a secure digital environment that connects hospitals, doctors, pharmacies, and patients through a unified platform. By replacing traditional paper-based prescriptions with digitally managed records, the MediSync system improves prescription accuracy, reduces the risk of fraud, and enhances communication between healthcare stakeholders.

The use of **React.js for the dynamic frontend interface** and **Node.js with Express.js for backend services** ensures efficient processing of system operations and smooth communication between modules. The **MongoDB database** provides a flexible and scalable solution for storing healthcare records, prescription data, and system logs. Through secure authentication using JWT tokens and role-based access control, the system maintains data integrity and protects sensitive medical information.

The implementation of MediSync demonstrates how modern web technologies can improve healthcare coordination and streamline prescription management processes. With proper system testing, modular architecture, and secure data handling, the platform ensures reliability, scalability, and a user-friendly experience for healthcare providers and patients.

5.1 Directions for Future Enhancements

Future enhancements of the MediSync platform can focus on expanding system capabilities and improving healthcare services through advanced technologies. The following improvements are proposed:

1. AI-Based Prescription Validation

Integrating artificial intelligence to automatically analyze prescriptions for potential medication conflicts, dosage errors, or unsafe drug combinations. This will help doctors and pharmacies improve patient safety and treatment accuracy.

2. Real-Time Notification System

Implementing real-time notifications using WebSockets or push notifications to instantly inform patients when prescriptions are created, updated, or verified by pharmacies.

3. Mobile Healthcare Application

Developing dedicated **Android and iOS mobile applications** using frameworks such as React Native. This will allow patients, doctors, and hospitals to access the MediSync platform conveniently from mobile devices.

4. Integration with Hospital Information Systems

Integrating MediSync with existing hospital management systems and electronic health record (EHR) platforms to allow seamless sharing of patient data and medical history.

5. Advanced Data Analytics and Reporting

Adding data analytics dashboards for hospitals and administrators to monitor prescription trends, pharmacy usage, and patient healthcare statistics for better decision-making.

6. Telemedicine and Online Consultation

Expanding the platform by adding video consultation features that allow patients to communicate with doctors remotely and receive digital prescriptions through the system.

7. Blockchain-Based Prescription Verification

Implementing blockchain technology to create tamper-proof prescription records, ensuring that medical prescriptions cannot be altered or forged.

Bibliography

BIBLIOGRAPHY

Web References

React.js

https://react.dev/
https://react.dev/learn
https://www.tutorialspoint.com/reactjs/index.htm
https://reactrouter.com/en/main

Node.js

https://nodejs.org/en/learn
https://nodejs.org/en/docs
https://www.tutorialspoint.com/nodejs/index.htm

MongoDB

https://www.mongodb.com/docs/
https://www.mongodb.com/atlas/database
https://www.tutorialspoint.com/mongodb/index.htm

Express.js

https://expressjs.com/
https://expressjs.com/en/starter/installing.html
https://www.tutorialspoint.com/expressjs/index.htm

Authentication & Security

https://jwt.io/introduction
https://www.npmjs.com/package/jsonwebtoken
https://www.npmjs.com/package/bcrypt

Other Development Tools

https://www.npmjs.com/
https://www.postman.com/
https://github.com/
https://code.visualstudio.com/

Book References

1. Learning React: A Hands-On Guide to Building Web Applications Using React and Redux

Author: **Kirupa Chinnathambi**

Publisher: **Addison-Wesley Professional**

Year: **2018**

Edition: **2nd Edition**

Description:

This book provides a strong foundation in React development including reusable components, state management, and building scalable user interfaces, which are essential for developing the frontend of the MediSync platform.

Link:

https://www.pearson.com/store/p/learning-react/P200000000179

---

2. Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node

Author: **Vasan Subramanian**

Publisher: **Apress**

Year: **2019**

Edition: **2nd Edition**

Description:

This book explains how to develop full-stack web applications using the MERN stack. It covers backend development with Node.js and Express, MongoDB database design, and integration with React applications.

Link:

https://link.springer.com/book/10.1007/978-1-4842-4391-6

---

3. Full-Stack React Projects: Learn MERN Stack Development by Building Modern Web Apps

Author: **Shama Hoque**

Publisher: **Packt Publishing**

Year: **2020**

Edition: **2nd Edition**

Description:

Provides practical examples of building modern web applications using the MERN stack and REST APIs, which are useful for implementing backend services and frontend integration in the MediSync system.

Link:

https://www.packtpub.com/en-us/product/full-stack-react-projects-9781788835534

---

4. The Road to React

Author: **Robin Wieruch**

Publisher: **Independently Published**

Year: **2020**

Edition: **4th Edition**

Description:

Focuses on React fundamentals, component-based architecture, and best practices for building scalable frontend applications.

Link:

https://www.roadtoreact.com/

---

5. MongoDB: The Definitive Guide

Authors: **Shannon Bradshaw, Eoin Brazil, Kristina Chodorow**

Publisher: **O'Reilly Media**

Year: **2019**

Edition: **3rd Edition**

Description:

Explains MongoDB architecture, schema design, indexing, and performance optimization, which are useful for managing user data, prescriptions, and healthcare records in the MediSync database.

Link:

https://www.oreilly.com/library/view/mongodb-the-definitive/9781491954454/

---

6. Node.js Design Patterns

Author: **Mario Casciaro & Luciano Mammino**

Publisher: **Packt Publishing**

Year: **2020**

Edition: **3rd Edition**

Description:

This book explains advanced backend development patterns using Node.js and helps developers design scalable server-side applications and RESTful APIs.

Link:

https://www.packtpub.com/product/node-js-design-patterns-third-edition/9781839214110

---

7. RESTful Web APIs

Authors: **Leonard Richardson & Mike Amundsen**

Publisher: **O'Reilly Media**

Year: **2013**

Description:

Provides a comprehensive guide to designing and implementing REST APIs, which are essential for building secure backend services used in the MediSync system.

Link:

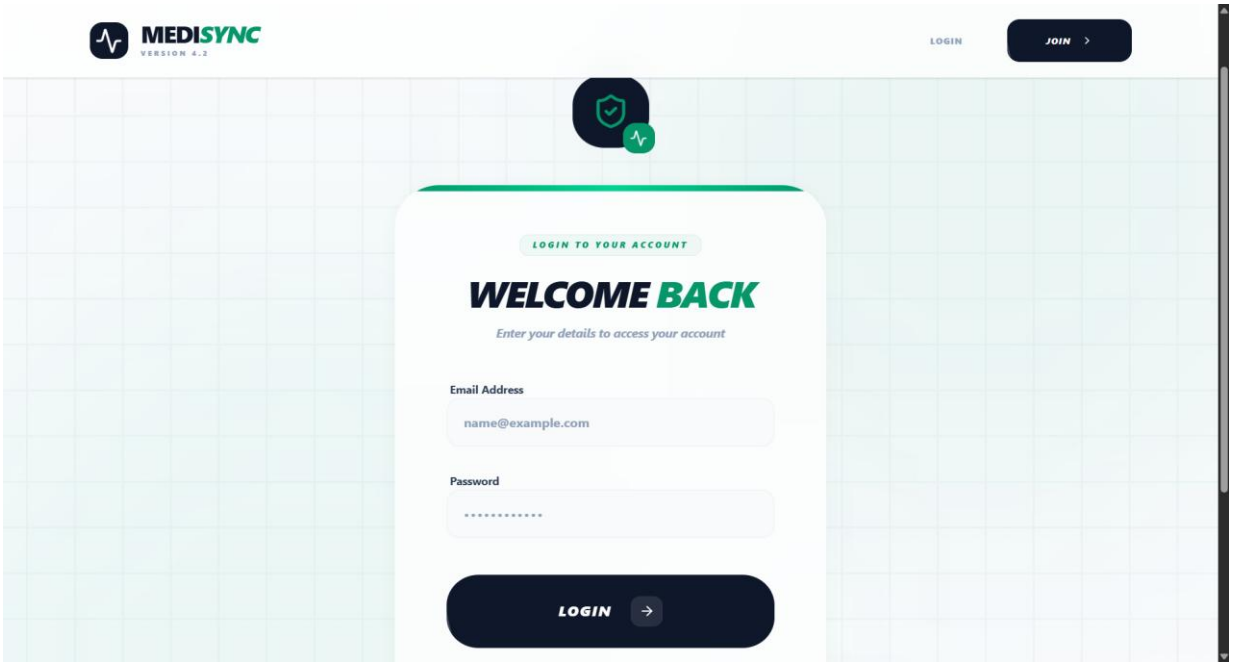
https://www.oreilly.com/library/view/restful-web-apis/9781449359713/

These references provide comprehensive knowledge about **MERN stack development, backend API design, database management, and frontend application architecture**, which were used in the development of the **MediSync healthcare synchronization platform**.

Annexures

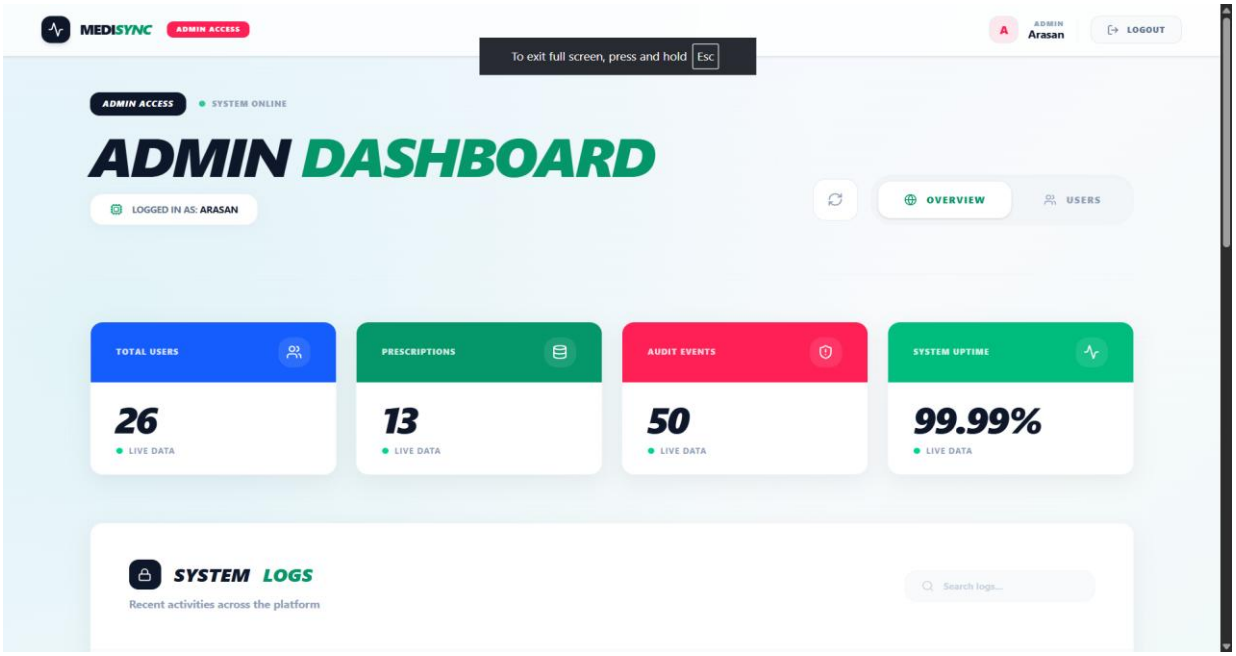
Annexure-A – Input Design

Login Screen:

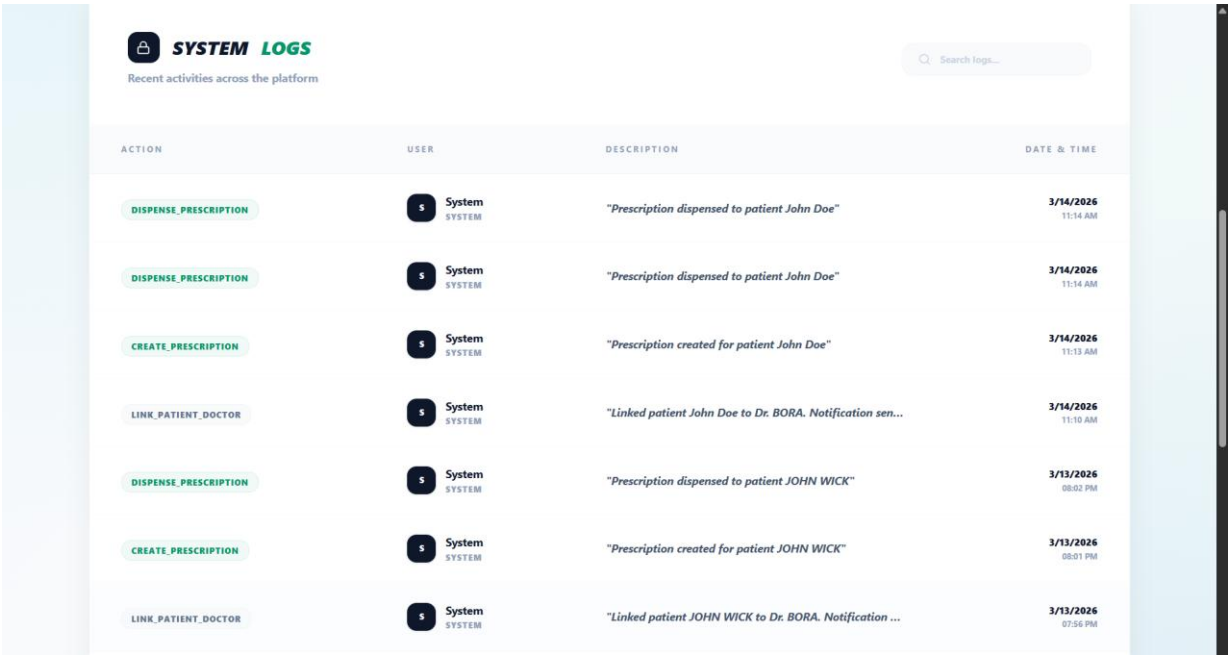


Annexure-B – Output Design

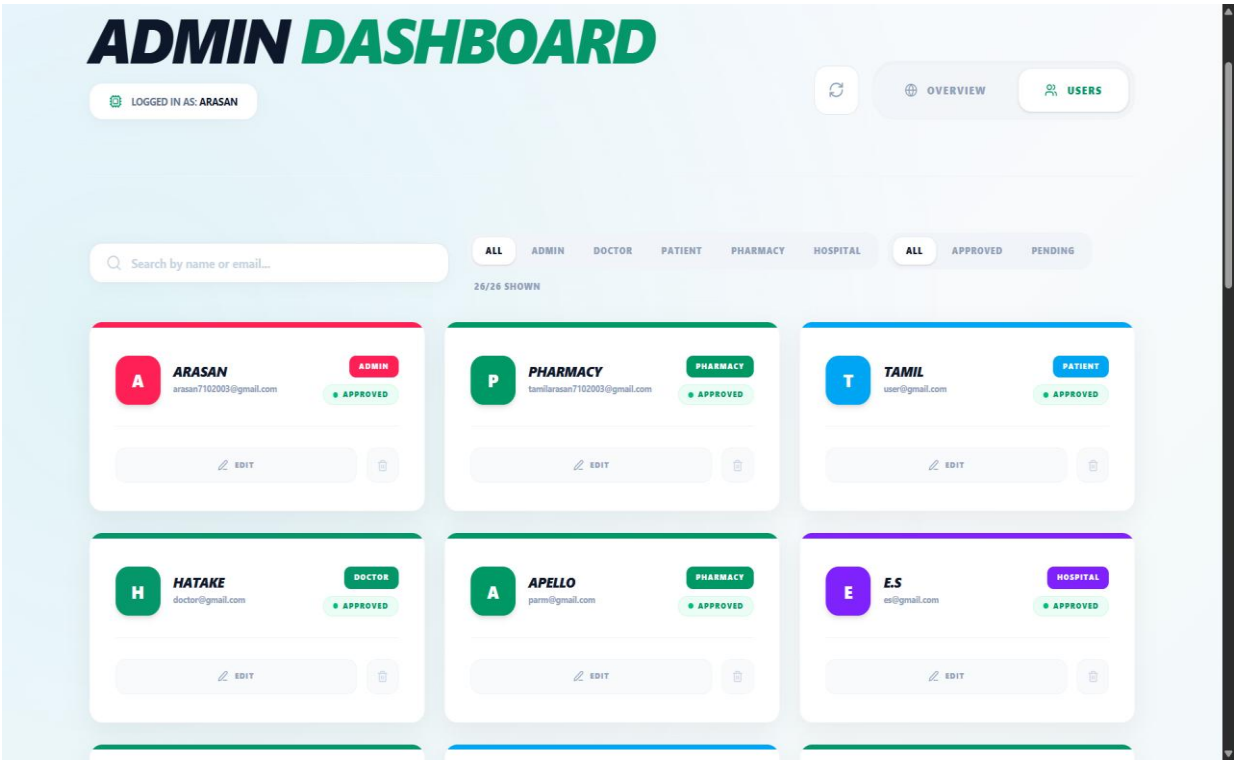
Admin Home Page:



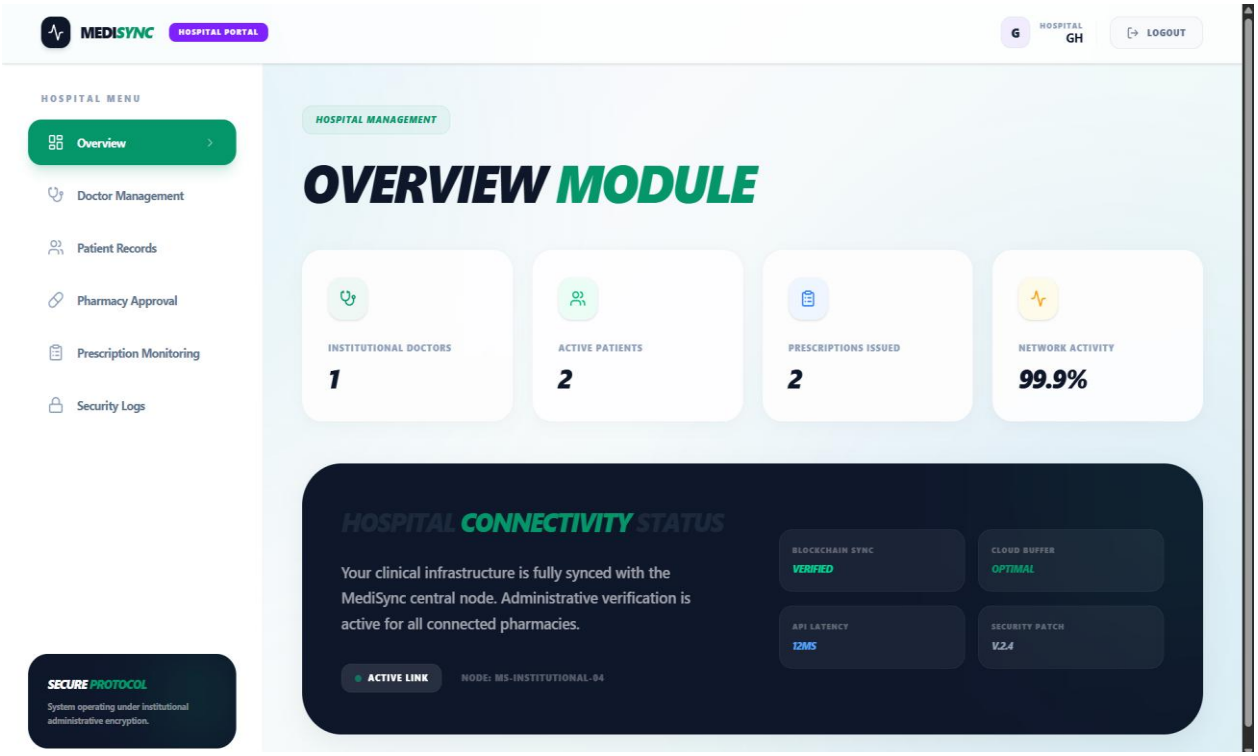
System Log Page:



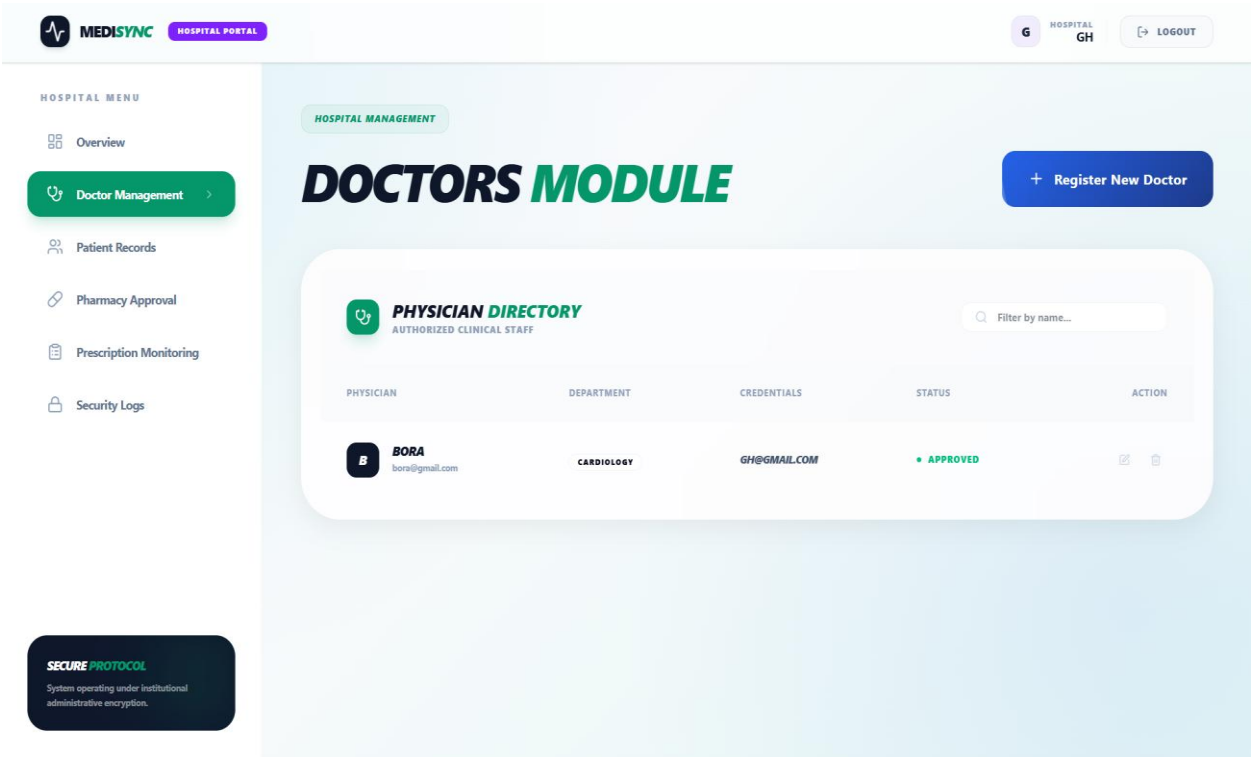
User Management Page:



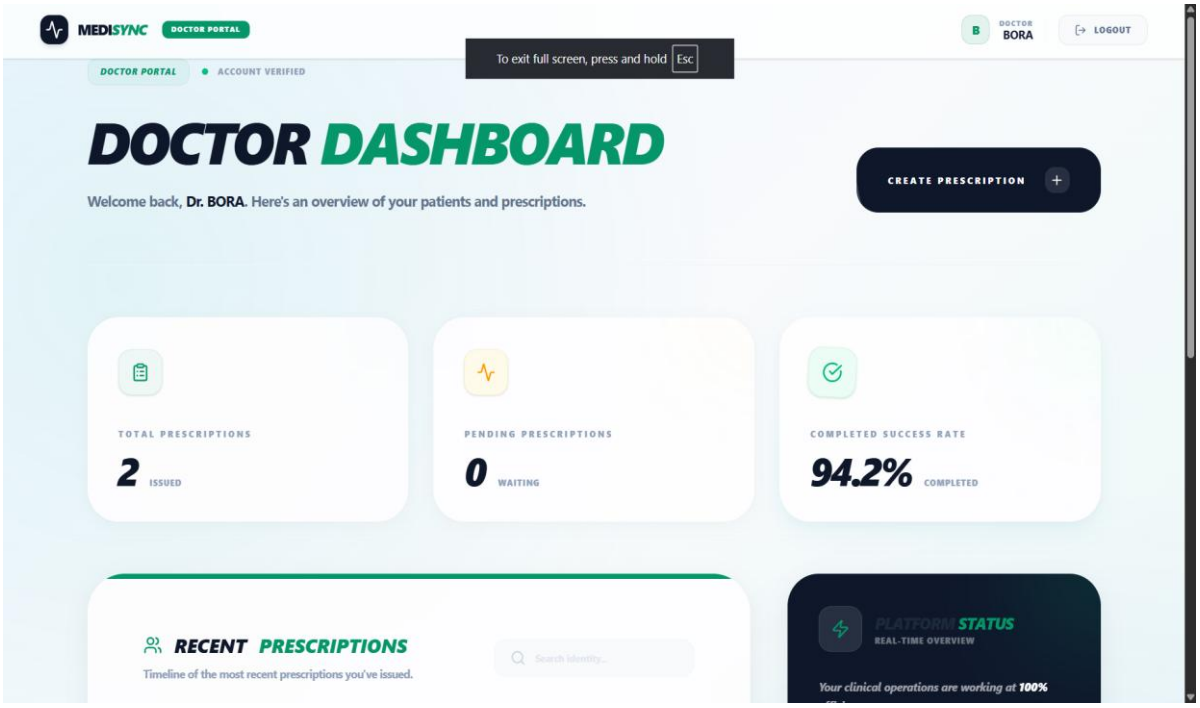
Hospital Home Page:



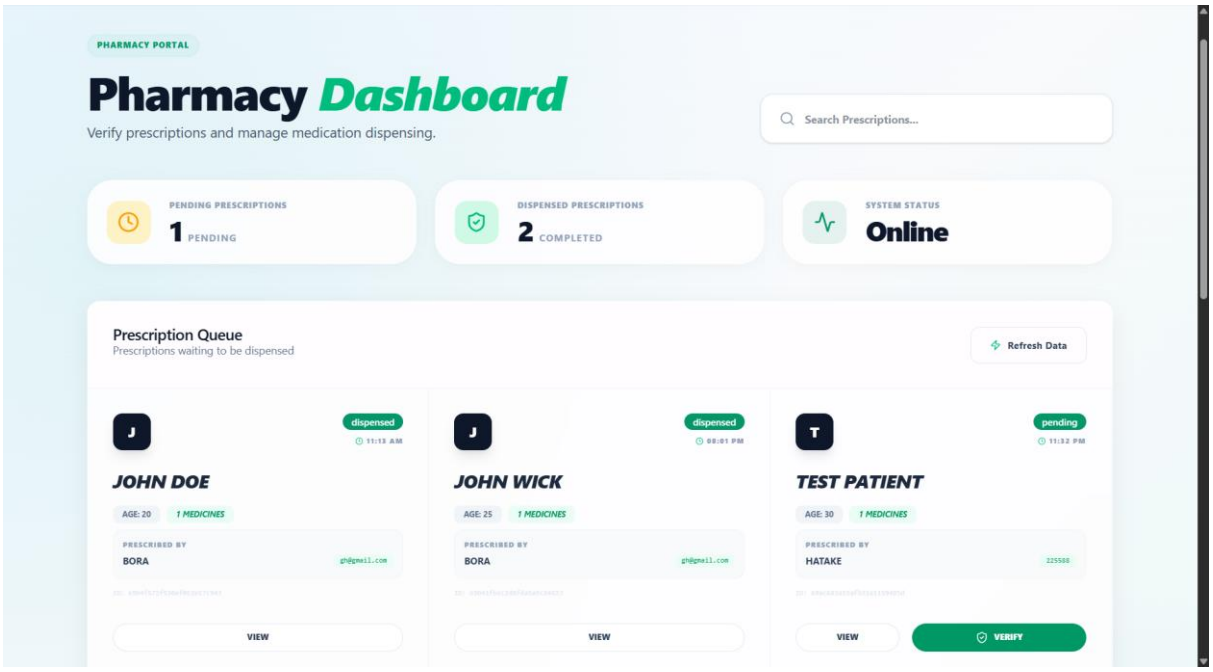
Docter Management Page:



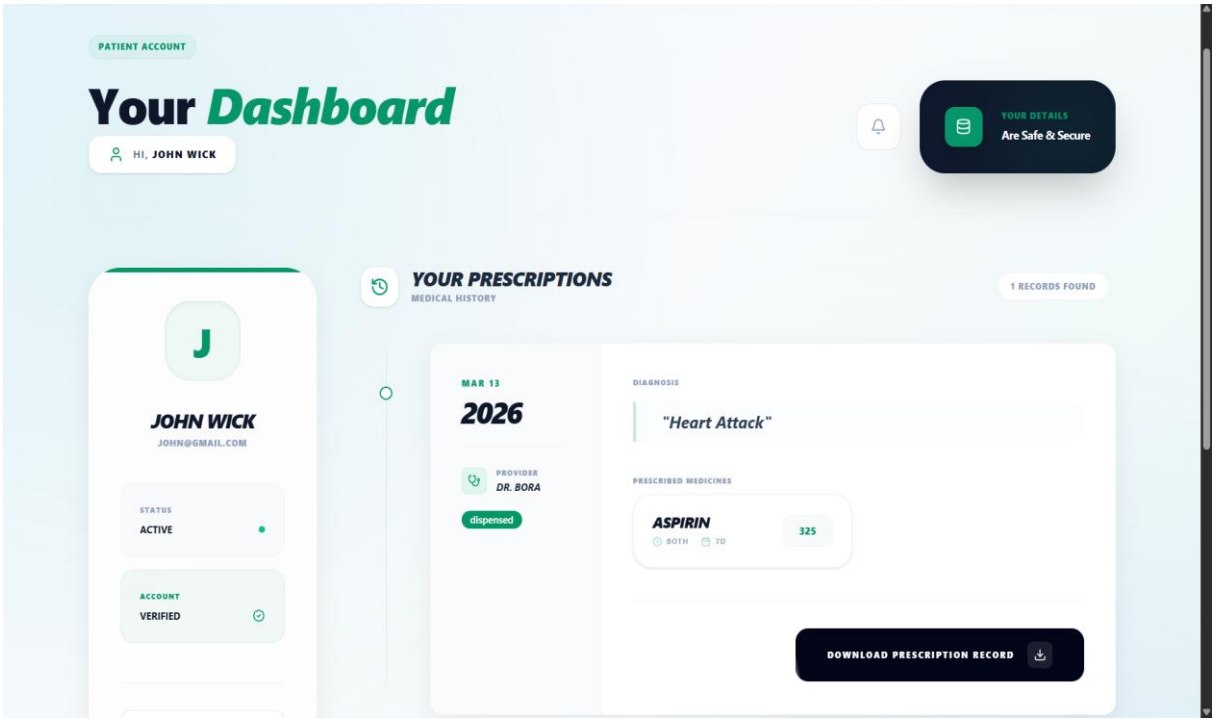
Docter Home Page:



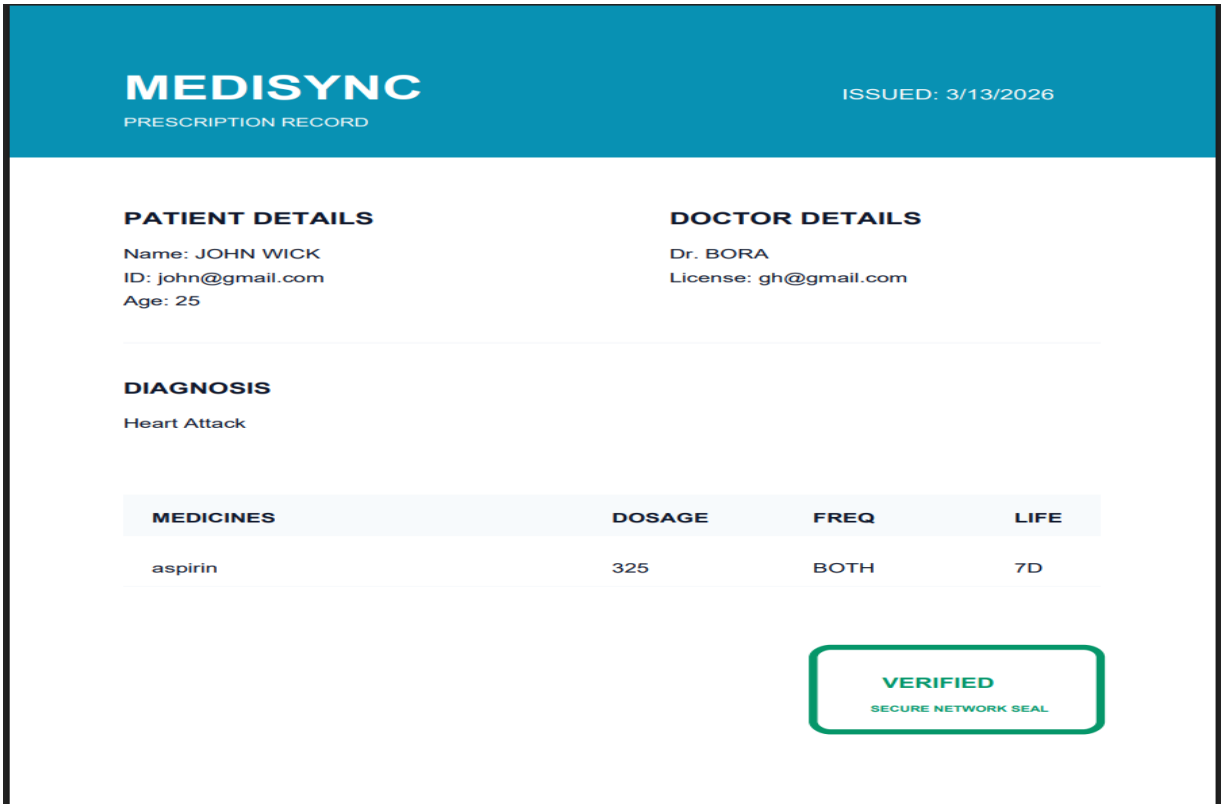
Pharmacy Home Page:



Patient Home Page:



Medisync Prescription Page:



Annexure-C – Source Code

FRONTEND:

App.jsx

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/Navbar';
import ProtectedRoute from './components/ProtectedRoute';
import { Toaster } from 'react-hot-toast';
import ErrorBoundary from './components/ErrorBoundary';

// Pages
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
const Home = () => <Navigate to="/dashboard" replace />;
function App() {
return (
<Router>
<AuthProvider>
<div className="min-h-screen bg-medical-bg flex flex-col font-sans transition-colors duration-500">
{/* 4K Constrained Container */}
<div className="w-full max-w-[2000px] mx-auto min-h-screen flex flex-col relative shadow-2xl bg-white/40 ring-1 ring-slate-900/5">
<Toaster position="top-right"
toastOptions={{
className: 'font-medium tracking-tight rounded-xl shadow-lg border border-slate-100',
style: { backdropFilter: 'blur(8px)', background: 'rgba(255, 255, 255, 0.9)', color: '#0f172a' }
}}
/>
<Navbar />
```

```
<main className="flex-1 w-full flex flex-col overflow-hidden">

<ErrorBoundary>
<Routes>

<Route path="/" element={<Home />} />
<Route path="/login" element={<Login />} />
<Route path="/register" element={<Register />} />
<Route element={<ProtectedRoute />}>
<Route path="/dashboard" element={<Dashboard />} />
</Route>
</Routes>
</ErrorBoundary>
</main>
</div>
</div>
</AuthProvider>
</Router>
);
}

export default App;
```

BACKEND :

index.js :

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');
const cookieParser = require('cookie-parser');
dotenv.config();
const app = express();
const PORT = process.env.PORT || 5000;
// Middleware
app.use(express.json());
app.use(cookieParser());
app.use(cors({
  origin: ['http://localhost:5173'], // Vite default port
  credentials: true
}));
// Database Connection
const promptConnection = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Connected Successfully');
  } catch (error) {

    console.error('MongoDB Connection Failed:', error.message);
  }
```

```
}
promptConnection();
// Routes Placeholder
app.get('/', (req, res) => {
   res.send('MediSync API is Running');
});
// Routes
const authRoutes = require('./routes/authRoutes');
const prescriptionRoutes = require('./routes/prescriptionRoutes');
const hospitalRoutes = require('./routes/hospitalRoutes');
app.use('/api/auth', authRoutes);
app.use('/api/prescriptions', prescriptionRoutes);
app.use('/api/hospital', hospitalRoutes);
app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
});
```